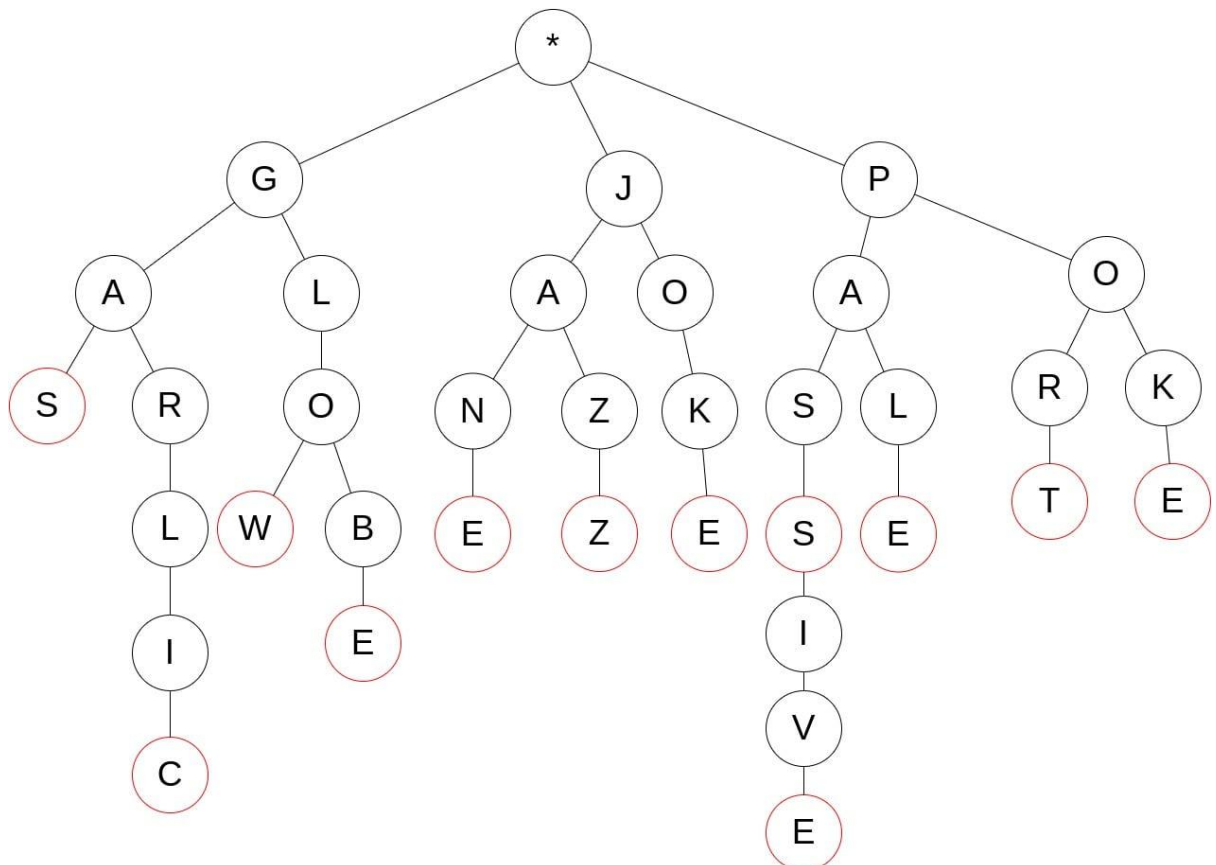


Cartopia – RI Project

- 1) **Autocomplete:** we implement autocomplete by using a trie(prefix tree) that stores strings character by character. Each node represents a prefix, and the children represent the next character



This is a better approach than scanning every catalog of product titles for each keystroke, and testing *startswith(prefix)*, which would have a complexity of $O(p \cdot L)$, where p -number of products, and L -average title length. With trie you walk at most m edges where $m = \text{len}(\text{prefix})$. Cost of the walk is $O(m)$ independent of how many products exist, as long as the trie is already built

In this project the trie is rebuilt on each API call (simple but not optimal for huge catalogs, which is fine for a small project like this). Complexity per request becomes $O(\text{sum of title lengths}) + O(m) + O(K)$ where K -how many product IDs are stored at the terminal node (bounded by limit when listing)

Algorithm

For each product title, the code walks from the root and adds the product's id to every node along the path (not only the leaf). Example: title laptop adds the id to nodes for l, la, lap, ..., laptop. Then prefix lap lands on the node after l, a, p and that node's `product_ids` set is exactly products whose title starts with lap.

Pseudocode:

```
build_trie(products):
    root = empty node
    for each product p:
        for each char c in lower(p.name):
            go to child c
            add p.id to current node.product_ids
    return root

suggest(prefix):
    node = root
    for c in lower(prefix):
        if c not in node.children: return []
        node = node.children[c]
    return up to limit products whose ids are in node.product_ids
```

Code implementation:

```
def build_trie(products):
    """
    Build a trie from all product titles (lowercased)

    For each product we walk the trie character by character and add that product's
    ID to every node along the path. So for example "laptop" adds the product to nodes
    for "l", "la", "lap", "lapt", "lapto", "laptop". That way when the user types
    "lap" we can follow the path and immediately get all products whose title
    starts with "lap" (e.g. "Laptop ASUS ...") without scanning all titles

    Returns the root node of the trie
    """
    root = TrieNode()
    for p in products:
        title_lower = (p.name or '').lower().strip()
        if not title_lower:
            continue
        node = root
        for char in title_lower:
            node = node.children[char]
            node.product_ids.add(p.id)
    return root
```

```

def get_suggestions(products, prefix, limit=10):
    """
    Return product suggestions whose title starts with "prefix" (case-insensitive)

    Steps:
    1. Build the trie from the given product list
    2. Walk from the root following each character of "prefix"
    3. If we hit a missing character, no product has that prefix -> return []
    4. Otherwise the node we land on has product_ids for all matching titles;
       we map those back to products and return up to "limit" items as
       [{id, title, slug}, ...]

    Note: We rebuild the trie on every call. TO DO: for very large catalogs we could
    cache the trie and invalidate when products change
    """
    if not products or not prefix or not prefix.strip():
        return []
    prefix = prefix.lower().strip()
    root = build_trie(products)
    node = root
    for char in prefix:
        if char not in node.children:
            return []
        node = node.children[char]
    product_by_id = {p.id: p for p in products}
    out = []
    for pid in node.product_ids:
        if len(out) >= limit:
            break
        p = product_by_id.get(pid)
        if p:
            out.append({'id': p.id, 'title': p.name, 'slug': p.slug})
    return out

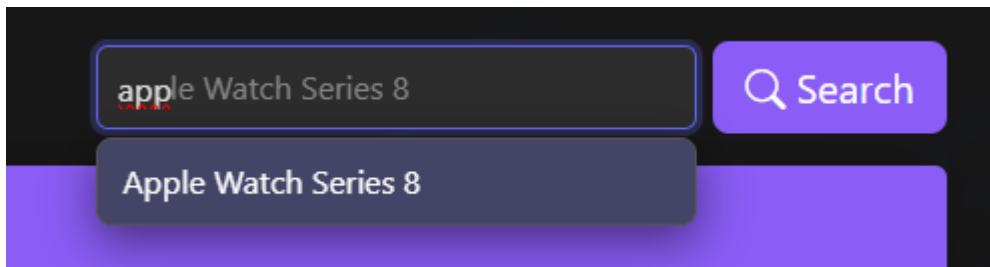

def autocomplete_api(request):
    """
    JSON API for search bar autocomplete. GET ?q=lap -> {suggestions: [{id, title, slug}, ...]}.
    Uses a trie over product titles so we only return products whose name starts with q.
    """
    q = (request.GET.get('q') or '').strip()[:80]
    if not q:
        return JsonResponse({'suggestions': []})
    products = list[Any](Product.objects.filter(is_active=True))
    suggestions = get_suggestions(products, q, limit=8)
    return JsonResponse({'suggestions': suggestions})

```

Complexity:

- Prefix walk: $O(m)$ with $m = |\text{prefix}|$
- Trie build for one title of length L : $O(L)$
- Full build over all titles: $O(\text{sum of title lengths})$

Project demo:



2) Product Similarity (TF-IDF + cosine):

We treat each product as a document built from name, description and specifications, then we map each document to a vector in a high-dimensional space (one dimension per vocabulary term, after vectorizer settings). TF-IDF assigns a weight to each term in each document. Cosine similarity compares two vectors by the cosine of the angle between them.

TD-IDF

$$w_{i,j} = tf_{i,j} \times \log \left(\frac{N}{df_i} \right)$$

$tf_{i,j}$ = number of occurrences of i in j
 df_i = number of documents containing i
 N = total number of documents

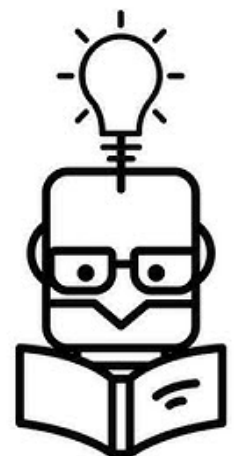
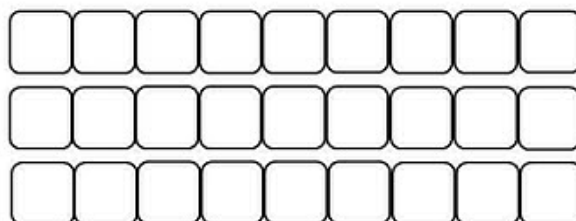
TF-IDF



$$tf(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

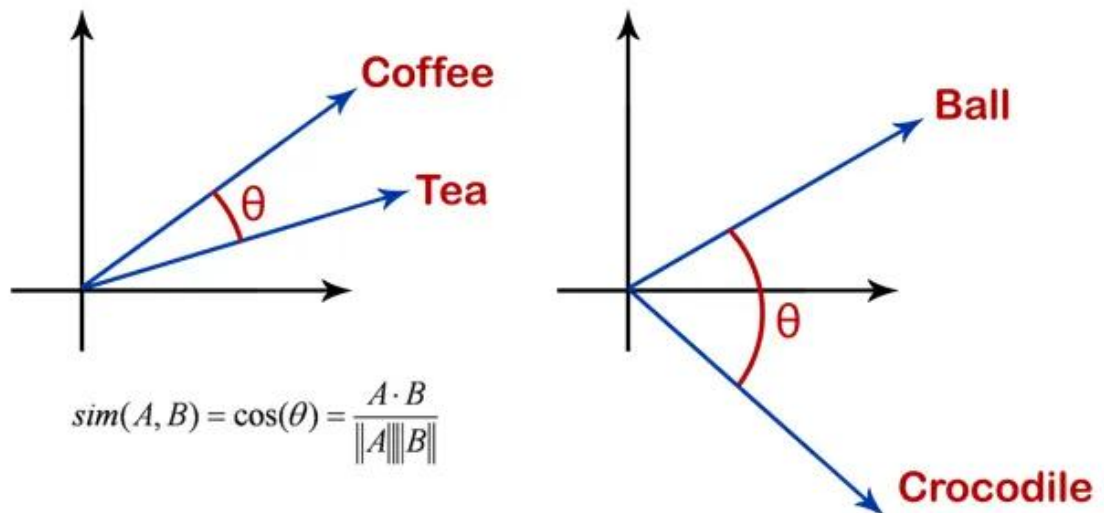


$$idf(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|}$$



- **TF (term frequency)**: if two products both mention “SSD”, “RAM”, “display”, they move closer in vector space
- **IDF (inverse document frequency)**: words that appear in almost every product (for example “with”) get lower weight; rare informative words get higher weight

Cosine similarity



Cosine measures direction, not raw length, so a long description does not automatically look “more similar” to everything just because it has more tokens (unlike naive dot product without normalization)

Pipeline in code:

1. `product_detail(request, slug)` loads current product and active catalog, then calls `get_similar_products(product, all_active, top_k=4)`

```

def product_detail(request, slug):
    product = get_object_or_404(Product, slug=slug, is_active=True)
    # similar products by text similarity (TF-IDF + cosine), not by category
    all_active = list[Any](Product.objects.filter(is_active=True))
    related_products = get_similar_products(product, all_active, top_k=4)

    in_wishlist = False
    in_cart = False
    cart_quantity = 0
    user_review = None
    can_review = False

    if request.user.is_authenticated:
        in_wishlist = Wishlist.objects.filter(user=request.user, product=product).exists()
        cart_item = Cart.objects.filter(user=request.user, product=product).first()
        if cart_item:
            in_cart = True
            cart_quantity = cart_item.quantity

        user_review = Review.objects.filter(user=request.user, product=product).first()
        has_ordered = Order.objects.filter(user=request.user, items__product=product).exists()
        can_review = has_ordered and not user_review

    reviews = Review.objects.filter(product=product, is_approved=True).order_by('-created_at')
    average_rating = reviews.aggregate(Avg('rating'))['rating__avg'] or 0
    rating_count = reviews.count()

    review_form = None
    if request.user.is_authenticated and can_review:
        review_form = ReviewForm()

```

2. `get_product_text(product)` concatenates name + description + specifications for each product

```

def get_product_text(product):
    """
    Concatenate all searchable text for a product into one string
    TF-IDF will tokenize this and assign weights; more text = more chance to match
    """
    parts = [
        product.name or '',
        product.description or '',
        product.specifications or '',
    ]
    return ' '.join(parts)

```

3. `texts = [get_product_text(p) for p in all_products]` builds one text document per active product
4. `TfidfVectorizer(stop_words='english', max_features=5000) + fit_transform(texts)` builds vocabulary and TF-IDF matrix
5. `cosine_similarity(tfidf)` computes the full $n \times n$ similarity matrix
6. Ranking in `get_similar_products`: select row for current index, sort descending by score, skip self/score ≤ 0 , return first top_k

```
def get_similar_products(product, all_products, top_k=4):
    """
    Return the top_k products most similar to "product" by text similarity

    Steps:
    1. Build one text "blob" per product (name + description + specifications)
    2. TF-IDF vectorizer: converts each "blob" to a vector of term weights
        - TF = how often the term appears in this document
        - IDF = how rare the term is across all documents (rare terms get higher weight)
        - stop_words = 'english' removes "the", "and", etc. max_features caps vocabulary size
    3. cosine_similarity(tfidf) gives a matrix: sim[i,j] = similarity between doc i and j
    4. We take the row for our product (idx), sort other products by score descending,
        skip self and zero/negative scores, and return the first top_k Product instances
    """

    if not all_products or len(all_products) < 2:
        return []

    texts = [get_product_text(p) for p in all_products]
    product_ids = [p.id for p in all_products]
    vectorizer = TfidfVectorizer(stop_words='english', max_features=5000)
    try:
        tfidf = vectorizer.fit_transform(texts)
    except Exception:
        return []

    # Shape: (n_products, n_features) -> (n_products, n_products) similarity matrix
    sim = cosine_similarity(tfidf)
    idx = product_ids.index(product.id)
    scores = list[tuple[int, Any]](enumerate[Any](sim[idx])) # (index, similarity_score) for each product
    scores.sort(key=lambda x: -x[1]) # sort by similarity score descending
    out = []
    for i, score in scores:
        if i == idx or score <= 0:
            continue
        out.append(all_products[i])
        if len(out) >= top_k:
            break
    return out
```

7. The returned list is rendered in the product page
via related_products (templates/shop/product_detail.html)

```
{% if related_products %}
<hr class="my-5">
<h3 class="mb-4">Similar products</h3>
<div class="row">
    {% for related in related_products %}
```

Project demo:


[Home](#)
[Products](#)
[Specifications Search](#)
[Categories](#)

[Login](#)
[Register](#)



Apple Watch Series 8

€400.00

In Stock (14 available)

Category: Accessories

Supplier: ElectroWorld

Delivery Method: Standard Mail (€8.00)

Description

Apple Watch Series 8 with Always-On Retina display, GPS, water resistant

Specifications

Display: Always-On Retina
GPS: Yes
Water Resistant: Yes (50m)
Battery: Up to 18 hours
Sensors: Heart rate, ECG

[Download specifications file \(PDF\)](#)

[Login to Purchase](#)

Similar products



iPad Air Tablet

€600.00

[View Details](#)



iPhone 14 Pro

€1000.00

[View Details](#)



Sony WH-1000XM4 Bluetooth Headphones

€240.00

[View Details](#)



Webcam Logitech C920

€90.00

[View Details](#)

3) Specification search (Apache Lucene Java + BM25)

We need to rank products by how well their specifications field matches a user query (full text)

The Lucene indexing architecture in this project is document-based and centered on product specifications PDF files. For each active product, the Python bridge sends a JSON payload to the Java CLI, and the Java side creates one Lucene Document per product. Each document stores the product identifier in a stored numeric field (id) and indexes the specification text in a searchable text field (spec). The analyzer (StandardAnalyzer) tokenizes and normalizes the specification content, while the index is created in memory with RAMDirectory for fast per-request execution. This design makes it straightforward to retrieve ranked hits and map them back to Django product objects after search.

Lucene score analysis is based on BM25, configured explicitly through BM25Similarity. Conceptually, BM25 gives higher scores to products whose specification text matches more query terms, especially terms that are informative (rarer across the collection), while also controlling for document length so very long specifications are not unfairly favored. In practical terms, a higher Lucene score means a

better lexical match between the user query and the indexed specification field. This is why the default descending sort order (score_order=desc) is interpreted as “best match first.”

A concrete example of query behavior is easy to observe with specification-oriented terms. For the query *snapdragon 120hz*, products whose spec field contains both tokens typically appear at the top with higher scores, while products containing only one token rank lower, and products containing none are not returned. In the UI, these results are shown together with numeric Lucene scores, which helps explain ranking decisions and demonstrates that the system is applying BM25-based relevance ordering rather than simple keyword filtering.

This implementation uses Apache Lucene Java directly inside a Java CLI module:

- analyzer: StandardAnalyzer
- index: RAMDirectory
- scorer: BM25Similarity
- query parsing: QueryParser

BM25 is a probabilistic ranking function. It scores a document D for query Q as a sum over query terms.

Intuition:

- More occurrences of a query term in a spec should help, but not linearly forever - > saturation (k1)
- Rare terms across the catalog should count more -> IDF
- Long specs should not automatically win -> length normalization (b and |D|/avgdl)

Formula (matches Lucene BM25 configuration)

The diagram shows the BM25 formula with several annotations in red text and lines pointing to specific parts of the formula:

- sum the scores for each query term**: Points to the summation symbol $\sum_{t \in Q}$.
- term frequency saturation trick**: Points to the fraction $\frac{f_{t,D} \cdot (k_1 + 1)}{f_{t,D} + k_1 \cdot (1 - b + b \cdot \frac{|D|}{avgdl})}$.
- adjust saturation curve based on document length**: Points to the denominator $f_{t,D} + k_1 \cdot (1 - b + b \cdot \frac{|D|}{avgdl})$.
- forget about this: it doesn't affect score relationships so Lucene took it out**: Points to the $(k_1 + 1)$ term in the numerator.
- probabilistic flavor of IDF: Lucene adds a 1 inside the log, making it basically the same as traditional IDF**: Points to the $+ 0.5$ in the numerator of the log term.

$$\text{score}(D, Q) = \sum_{t \in Q} \frac{f_{t,D} \cdot (k_1 + 1)}{f_{t,D} + k_1 \cdot (1 - b + b \cdot \frac{|D|}{avgdl})} \cdot \log \frac{N - n_t + 0.5}{n_t + 0.5}$$

Lucene uses BM25 defaults unless explicitly changed. In this project we instantiate BM25Similarity() directly in Java.

Index architecture (what is stored)

Structure	Meaning
Lucene Document id (stored field)	product id used for mapping result back to Django product
Lucene Text field spec	analyzed specification text
Lucene inverted index posting lists	Token -> matching documents with term stats
Lucene BM25 scoring stats	internal DF, field length norms, collection stats

Note: the index is built per request in memory using Lucene RAMDirectory, which is still direct Lucene Java usage

Pipeline in code:

1. `specification_search(request)` reads `q` and `score_order`, loads active products, and calls `search_specifications_lucene_style(products, query, score_order)`

```
def specification_search(request):
    query = (request.GET.get('q') or '').strip()
    score_order = (request.GET.get('score_order') or 'desc').lower()
    if score_order not in {'asc', 'desc'}:
        score_order = 'desc'

    products = list[Any](Product.objects.filter(is_active=True))
    results = []
    if query:
        results = search_specifications_lucene_style(
            products,
            query,
            score_order=score_order,
        )

    return render(request, 'shop/specification_search.html', {
        'query': query,
        'score_order': score_order,
        'results': results,
        'lucene_scoring_note': get_lucene_scoring_note(),
    })
```

2. `search_specifications_lucene_style(...)` builds `documents = [{"id": ..., "text": product.specifications}, ...]` and calls `_run_lucene_cli(...)` (`shop/lucene_spec_search.py`)

```

def search_specifications_lucene_style(products, query_text, score_order="desc"):
    normalized_query = (query_text or "").strip()
    if not normalized_query:
        return []

    documents = [
        {
            "id": product.id,
            "text": _extract_spec_text_for_lucene(product),
        }
        for product in products
    ]
    lucene_rows = _run_lucene_cli(documents, normalized_query, score_order)

    product_by_id = {product.id: product for product in products}
    results = []
    for row in lucene_rows:
        product = product_by_id.get(int(row["id"]))
        if not product:
            continue
        results.append(
            {
                "product": product,
                "score": float(row["score"]),
            }
        )
    return results

```

3. `_run_lucene_cli(...)` writes input JSON, executes `java -jar lucene-java/target/lucene-search-cli.jar <input> <output>`, then reads output JSON

```

def _run_lucene_cli(documents, query_text, score_order):
    if not LUCENE_JAR_PATH.exists():
        raise RuntimeError(
            "Lucene Java jar was not found. Build it with: "
            "mvn -f lucene-java/pom.xml clean package"
        )

    payload = {
        "query": query_text,
        "score_order": score_order,
        "documents": documents,
    }

    with tempfile.TemporaryDirectory() as tmp_dir:
        tmp_path = Path(tmp_dir)
        input_path = tmp_path / "lucene-input.json"
        output_path = tmp_path / "lucene-output.json"
        input_path.write_text(json.dumps(payload), encoding="utf-8")

        # call the Java Lucene CLI to index and score this query
        subprocess.run(
            [
                "java",
                "-jar",
                str(LUCENE_JAR_PATH),
                str(input_path),
                str(output_path),
            ],
            cwd=str(ROOT_DIR),
            check=True,
            capture_output=True,
            text=True,
        )

    if not output_path.exists():
        return []
    return json.loads(output_path.read_text(encoding="utf-8"))

```

4. In Java, LuceneSearchCli.main(...) reads payload and calls runLuceneSearch(docs, queryText, scoreOrder)

```

public static void main(String[] args) throws Exception {
    if (args.length != 2) {
        throw new IllegalArgumentException("Usage: java -jar lucene-search-cli.jar <input-json-path> <output-json-path>");
    }

    Path inputPath = Path.of(args[0]);
    Path outputPath = Path.of(args[1]);
    String rawInput = Files.readString(inputPath);

    Map<String, Object> payload = MAPPER.readValue(rawInput, new TypeReference<>() {});
    String queryText = ((String) payload.getOrDefault("query", "")).trim();
    String scoreOrder = ((String) payload.getOrDefault("score_order", "desc")).trim().toLowerCase();
    List<Map<String, Object>> docs = (List<Map<String, Object>>) payload.getOrDefault("documents", new ArrayList<>());

    List<Map<String, Object>> results = runLuceneSearch(docs, queryText, scoreOrder);
    MAPPER.writeValue(outputPath.toFile(), results);
}

```

5. runLuceneSearch(...) creates StandardAnalyzer, RAMDirectory, IndexWriterConfig, and sets BM25Similarity()

```

private static List<Map<String, Object>> runLuceneSearch(List<Map<String, Object>> docs, String queryText, String scoreOrder) {
    if (queryText.isBlank() || docs.isEmpty()) {
        return Collections.emptyList();
    }

    StandardAnalyzer analyzer = new StandardAnalyzer();
    Directory directory = new RAMDirectory();
    IndexWriterConfig config = new IndexWriterConfig(analyzer);
    config.setSimilarity(new BM25Similarity());
}

```

6. For each document, Java indexes one Lucene Document with StoredField("id", ...) and TextField("spec", text, Field.Store.NO) (LuceneSearchCli.java)

```

try (IndexWriter writer = new IndexWriter(directory, config)) {
    for (Map<String, Object> row : docs) {
        int id = ((Number) row.get("id")).intValue();
        String text = String.valueOf(row.getOrDefault("text", ""));

        Document doc = new Document();
        doc.add(new StoredField("id", id));
        doc.add(new TextField("spec", text, Field.Store.NO));
        writer.addDocument(doc);
    }
}

```

7. Query phase uses QueryParser("spec", analyzer) + searcher.search(query, docs.size()); each ScoreDoc becomes {id, score}

```

List<Map<String, Object>> results = new ArrayList<>();
try (DirectoryReader reader = DirectoryReader.open(directory)) {
    IndexSearcher searcher = new IndexSearcher(reader);
    searcher.setSimilarity(new BM25Similarity());

    QueryParser parser = new QueryParser("spec", analyzer);
    Query query = parser.parse(QueryParser.escape(queryText));
    TopDocs topDocs = searcher.search(query, docs.size());

    for (ScoreDoc scoreDoc : topDocs.scoreDocs) {
        Document doc = searcher.doc(scoreDoc.doc);
        int productId = doc.getField("id").numericValue().intValue();
        float score = scoreDoc.score;
        if (score <= 0f) {
            continue;
        }

        Map<String, Object> item = new HashMap<>();
        item.put("id", productId);
        item.put("score", score);
        results.add(item);
    }
} finally {
    directory.close();
}

```

8. Python maps returned ids to product objects and returns [{product, score}, ...]; the view renders results and score values in specification_search.html

```

{% if query %}
  <div class="mb-3">
    <strong>{{ results|length }}</strong> result(s) for <strong>{{ query }}</strong>
  </div>
  {% if results %}
    <div class="row">
      {% for item in results %}
        {% with product=item.product %}
          <div class="col-md-6 mb-4">
            <div class="card h-100">
              <div class="card-body">
                <h5 class="card-title">
                  <a href="{% url 'product_detail' product.slug %}" class="text-decoration-none">{{ product.
                </h5>
                <p class="text-muted mb-2">Lucene score: {{ item.score|floatformat:5 }}</p>
                <p class="mb-0">{{ product.specifications|truncatewords:35 }}</p>
              </div>
            </div>
          </div>
        {% endwith %}
      {% endfor %}
    </div>
  {% else %}
    <div class="alert alert-info">No specification documents matched this query</div>
  {% endif %}
{% endif %}
{% endblock %}

```

Project demo:



Advanced specification search

Specification text

Example: resolution 1900x1200

Sort by Lucene score

Descending

[Search](#)

[Show Lucene scoring info](#)

Advanced specification search

Specification text

RAM

Sort by Lucene score

Descending

[Search](#)

[Show Lucene scoring info](#)

How Lucene score works

Apache Lucene Java computes BM25 scores over indexed specification text. Higher score means a better match for the query

- Higher score means the specification is a better match
- Descending sort puts the best matches first
- Ascending sort puts the weaker matches first

3 result(s) for **RAM**

Laptop ASUS VivoBook 15

Lucene score: 0.70104

Processor: Intel Core i5-1135G7 RAM: 8GB DDR4 Storage: 512GB SSD
Display: 15.6" Full HD Graphics: Intel Iris Xe

Laptop Gaming ASUS ROG Strix

Lucene score: 0.69315

Processor: AMD Ryzen 9 5900HX Graphics: NVIDIA RTX 3070 RAM: 16GB
DDR4 Storage: 1TB SSD Display: 15.6" Full HD 144Hz

Samsung Galaxy S23 Ultra

Lucene score: 0.49699

Display: 6.8" Dynamic AMOLED 2X Processor: Snapdragon 8 Gen 2 RAM:
12GB Storage: 256GB Camera: 200MP + 10MP + 10MP + 12MP